# Architektura i stos technologiczny dla zdecentralizowanego bukmachera w modelu hybrydowym (Pari-Mutuel + Yield-Bearing) na sieci Solana

Zespół Deweloperski

3 października 2026

**Streszczenie**

Niniejszy dokument stanowi kompleksowe wytyczne architektoniczne i technologiczne dla zespołu deweloperskiego budującego zdecentralizowanego bukmachera na sieci Solana. Wykorzystujemy hybrydowy model łączący zakłady Pari-Mutuel z generowaniem zysku (Yield-Bearing) poprzez integrację z protokołami DeFi. Dokument opisuje stos technologiczny, model kont w Anchorze, główny przepływ logiki, krytyczne zabezpieczenia oraz strategię na hackathon. Ma służyć jako fundament implementacji projektu.

## Spis treści

**1 Wprowadzenie** **3**

**2 Stos technologiczny (Tech Stack)** **3**
- 2.1 Smart Contract (Program on-chain) . . . 3
- 2.2 Frontend (Klient) . . . 3
- 2.3 Integracja Web3 (Frontend) . . . 3
- 2.4 Zarządzanie stanem klienta . . . 3
- 2.5 Wyrocznie (Oracles) . . . 3
- 2.6 Moduł DeFi (Yield) . . . 3

**3 Architektura Kont (Account Model w Anchorze)** **3**
- 3.1 MarketState (PDA) . . . 4
- 3.2 Vault (Token Account PDA) . . . 4
- 3.3 UserBetReceipt (PDA) . . . 4

**4 Główny przepływ logiki (Execution Flow)** **5**
- 4.1 initialize_market . . . 5
- 4.2 place_bet . . . 5
- 4.3 resolve_market . . . 5
- 4.4 claim_winnings . . . 6

**5 Wytyczne krytyczne (Zabezpieczenia i edge-cases)** **6**
- 5.1 Brak płynności / Anulowanie . . . 6
- 5.2 Precyzja matematyczna . . . 6
- 5.3 Security . . . 6

**6 Strategia na hackathon (Devnet)** **6**

**7 Podsumowanie** **7**

# 1 Wprowadzenie

Projekt zakłada stworzenie zdecentralizowanego bukmachera działającego w modelu hybrydowym: Pari-Mutuel (zakłady wzajemne) połączone z generowaniem zysku (Yield-Bearing). Całość opiera się na specyfice sieci Solana: równoległym przetwarzaniu transakcji, niskich opłatach oraz kompozycyjności (CPI) protokołów DeFi. Klasyczny bukmacher zostaje zastąpiony logiką on-chain, co eliminuje pośrednika i zwiększa transparentność.

# 2 Stos technologiczny (Tech Stack)

## 2.1 Smart Contract (Program on-chain)

- **Język:** Rust
- **Framework:** Anchor – absolutny standard, który zdejmuje z zespołu konieczność ręcznej serializacji kont i zapewnia wbudowane mechanizmy bezpieczeństwa.

## 2.2 Frontend (Klient)

- **Framework:** Next.js (React)
- **Styling:** TailwindCSS

## 2.3 Integracja Web3 (Frontend)

- `@solana/web3.js` v1 (lub v2, jeśli zespół czuje się pewnie z nowym API)
- `@solana/wallet-adapter-react` do obsługi portfeli
- Do interakcji z programem używamy biblioteki klienta generowanej przez Anchora (`@coral-xyz/anchor`)

## 2.4 Zarządzanie stanem klienta

- React Query (TanStack Query) do odpytywania RPC o stan kont (fetching kont PDA) – zapobiega to blokowaniu UI i ułatwia cache'owanie.

## 2.5 Wyrocznie (Oracles)

- Switchboard v3 (Custom Functions/HTTP). W przypadku zakładów sportowych (np. wyniki CS2, piłka nożna) potrzebujemy elastyczności, aby uderzyć w konkretne API (np. HLTV, API-Football). Switchboard pozwala na napisanie prostej funkcji off-chain (np. w TypeScript), która weryfikuje wynik i wrzuca go na łańcuch.

## 2.6 Moduł DeFi (Yield)

- Kamino Finance (Lending) lub Marginfi.

# 3 Architektura Kont (Account Model w Anchorze)

Na Solanie stan nie jest przechowywany w kodzie, ale na oddzielnych kontach przypisanych do programu. Wymagane są następujące struktury oparte na PDA (Program Derived Addresses):

### 3.1 MarketState (PDA)

Główne konto przechowujące metadane konkretnego meczu.

- **Seed:** `["market", match_id]`
- **Pola:**
  - `match_id: u64`
  - `start_time: i64`
  - `end_time: i64`
  - `oracle_feed_pubkey: Pubkey`
  - `pool_team_a: u64`
  - `pool_team_b: u64`
  - `status: MarketStatus` (Open, Locked, Resolved, Cancelled)
  - `winning_team: Option<u8>`

```
#[account]
pub struct MarketState {
    pub match_id: u64,
    pub start_time: i64,
    pub end_time: i64,
    pub oracle_feed_pubkey: Pubkey,
    pub pool_team_a: u64,
    pub pool_team_b: u64,
    pub status: MarketStatus,
    pub winning_team: Option<u8>,
}

#[derive(AnchorSerialize, AnchorDeserialize, Clone, PartialEq, Eq)]
pub enum MarketStatus {
    Open,
    Locked,
    Resolved,
    Cancelled,
}
```

Listing 1: Struktura konta MarketState

### 3.2 Vault (Token Account PDA)

Skarbiec przechowujący USDC przypisany do danego rynku. Autorytetem tego skarbca jest program.

- **Seed:** `["vault", market_pubkey]`
- **Typ:** Token Account (USDC)
- **Authority:** Program (PDA)

### 3.3 UserBetReceipt (PDA)

Dowód postawienia zakładu przez konkretnego użytkownika.

- **Seed:** `["receipt", market_pubkey, user_pubkey]`
- **Pola:**

- `user_pubkey: Pubkey`
- `team_selected: u8`
- `amount_staked: u64`
- `claimed: bool`

```rust
#[account]
pub struct UserBetReceipt {
    pub user_pubkey: Pubkey,
    pub team_selected: u8,
    pub amount_staked: u64,
    pub claimed: bool,
}
```

Listing 2: Struktura konta UserBetReceipt

# 4 Główny przepływ logiki (Execution Flow)

Kod należy rozpisać na cztery główne instrukcje (Instructions) w Anchorze:

## 4.1 initialize_market

Admin lub skrypt (crank) inicjuje rynek. Tworzone jest konto `MarketState` oraz powiązany z nim `Vault` na USDC. Ustawiany jest adres wyroczni Switchboard, której program będzie ufał przy rozwiązaniu.

## 4.2 place_bet

Użytkownik wywołuje tę instrukcję, przesyłając USDC ze swojego portfela do skarbca (`Vault`).

1. Program weryfikuje, czy aktualny czas sieciowy (`Clock::get()`) jest wcześniejszy niż czas rozpoczęcia meczu (`start_time`). Jeśli mecz trwa, transakcja zostaje odrzucona (zabezpieczenie przed grą na gotowy wynik).
2. Program aktualizuje pulę `pool_team_a` lub `pool_team_b` w `MarketState` i tworzy/aktualizuje `UserBetReceipt`.
3. **Opcja CPI (Yield):** W tym samym bloku kodu program wykonuje Cross-Program Invocation do protokołu lendingowego (np. Kamino), wpłacając zebrane USDC na depozyt oprocentowany.

## 4.3 resolve_market

Po zakończeniu meczu, wyrocznia (Switchboard) aktualizuje swoje konto na łańcuchu. Ktokolwiek (tzw. permissionless crank) wywołuje tę instrukcję.

1. Program odczytuje dane z wyroczni, ustawia pole `winning_team` i blokuje rynek (`status = Resolved`).
2. Jeśli środki były w Kamino, program wywołuje CPI wyciągające kapitał główny + wygenerowane odsetki.

### 4.4 claim_winnings

Zwycięski użytkownik zgłasza się po nagrodę.

1. Program sprawdza `UserBetReceipt` i upewnia się, że wytypowana drużyna zgadza się z `winning_team`.
2. **Matematyka Pari-Mutuel:** Wypłata to

$$Wypata = \left(\frac{UserStake}{WinningTeamPool}\right) \times TotalPool$$

   Brak marży.
3. Program wysyła należną kwotę USDC do użytkownika i oznacza `claimed = true`.
4. Wygenerowany yield jest wysyłany na osobne konto `Treasury` jako zysk twórców platformy.

## 5 Wytyczne krytyczne (Zabezpieczenia i edge-cases)

### 5.1 Brak płynności / Anulowanie

Zespół musi zaprogramować stan, w którym wyrocznia zwraca błąd (np. mecz odwołany) lub mecz zakończył się remisem (jeśli format na to pozwala). W takim przypadku status przechodzi w `Cancelled`, a użytkownicy mogą wywołać `refund`, aby odzyskać równe 100% wkładu (yield wraca do Treasury).

### 5.2 Precyzja matematyczna

Solana używa typu `u64` dla tokenów, które mają własne miejsca po przecinku (np. USDC ma 6). Zwróćcie uwagę zespołowi na potencjalne błędy zaokrągleń przy dzieleniu w modelu Pari-Mutuel. Zawsze zaokrąglajcie ułamkowe resztki w dół na korzyść protokołu (zapobiega to wyczerpaniu skarbca).

### 5.3 Security

Program musi bezwzględnie weryfikować autorytet wyroczni. Kontrakt nie może ufać dowolnemu kontu przekazanemu z frontendu jako wyrocznia – adres `oracle_feed_pubkey` musi być zahardkodowany na etapie inicjalizacji rynku i restrykcyjnie sprawdzany podczas zamykania.

## 6 Strategia na hackathon (Devnet)

Integracja prawdziwego CPI do Kamino na devnecie potrafi zablokować zespół na kilka dni z powodu różnic w adresach i niedostępności płynności testowej. Polecamy zespołowi zbudowanie **Mock Lending Program**.

- Niech stworzą drugi, banalnie prosty smart contract obok głównego.
- Główny kontrakt wyśle tam USDC, a Mock przy wypłacie zwróci wpłaconą kwotę + sztucznie wygenerowane 5% odsetek (wymintowane z lokalnego kranu).
- To wystarczy, by udowodnić sędziom działanie kompozycyjności DeFi (CPI) i podziału środków bez grzęźnięcia w problemach infrastruktury obcych protokołów.
- Wykażecie tym samym, że w pełni rozumiecie, jak technicznie usuwa się pośrednika z transakcji finansowej, a jednocześnie zachowujecie kontrolę nad środowiskiem demonstracyjnym.

# 7 Podsumowanie

Dokument ten stanowi kompletny fundament do implementacji zdecentralizowanego bukmachera na Solanie. Zespół powinien ściśle przestrzegać opisanych wytycznych, szczególnie w zakresie bezpieczeństwa i precyzji matematycznej. Strategia hackathonowa z Mock Lending Program pozwoli na szybkie dostarczenie działającego prototypu i zaprezentowanie kluczowych cech systemu.